

TF4507 Distributed Systems INTRODUCTION TO DISTRIBUTED SYSTEMS

Henry Novianus Palit (hnpalit@petra.ac.id) – INFORMATICS Department



REVOLUTION OF COMPUTER SYSTEMS *[1]*

- From 1945 (the beginning of the modern computer era) until about 1985, computers were large and expensive
 - A minicomputer costed at least tens of thousands of dollars
 - Most organizations had only a handful of computers and those computers operated independently from one another
- In the mid-1980s, two advances in technology brought a revolution to the computer systems



REVOLUTION OF COMPUTER SYSTEMS [2]

- 1st development: powerful microprocessors
 - The first IBM PC was built using Intel 8088 and commercial components; it was released in 1981 and became the best-selling computer in history
 - Uncharacteristically IBM published the complete plans in a book sold for \$49, and numerous other companies began making the PC clone
 - 8-bit machines → 16-bit machines → 32-bit machines → 64-bit machines
 - Many of these machines had the computing power of a mainframe (i.e., large) computer, but with a fraction ($1/1000^{\text{th}}$) of the price



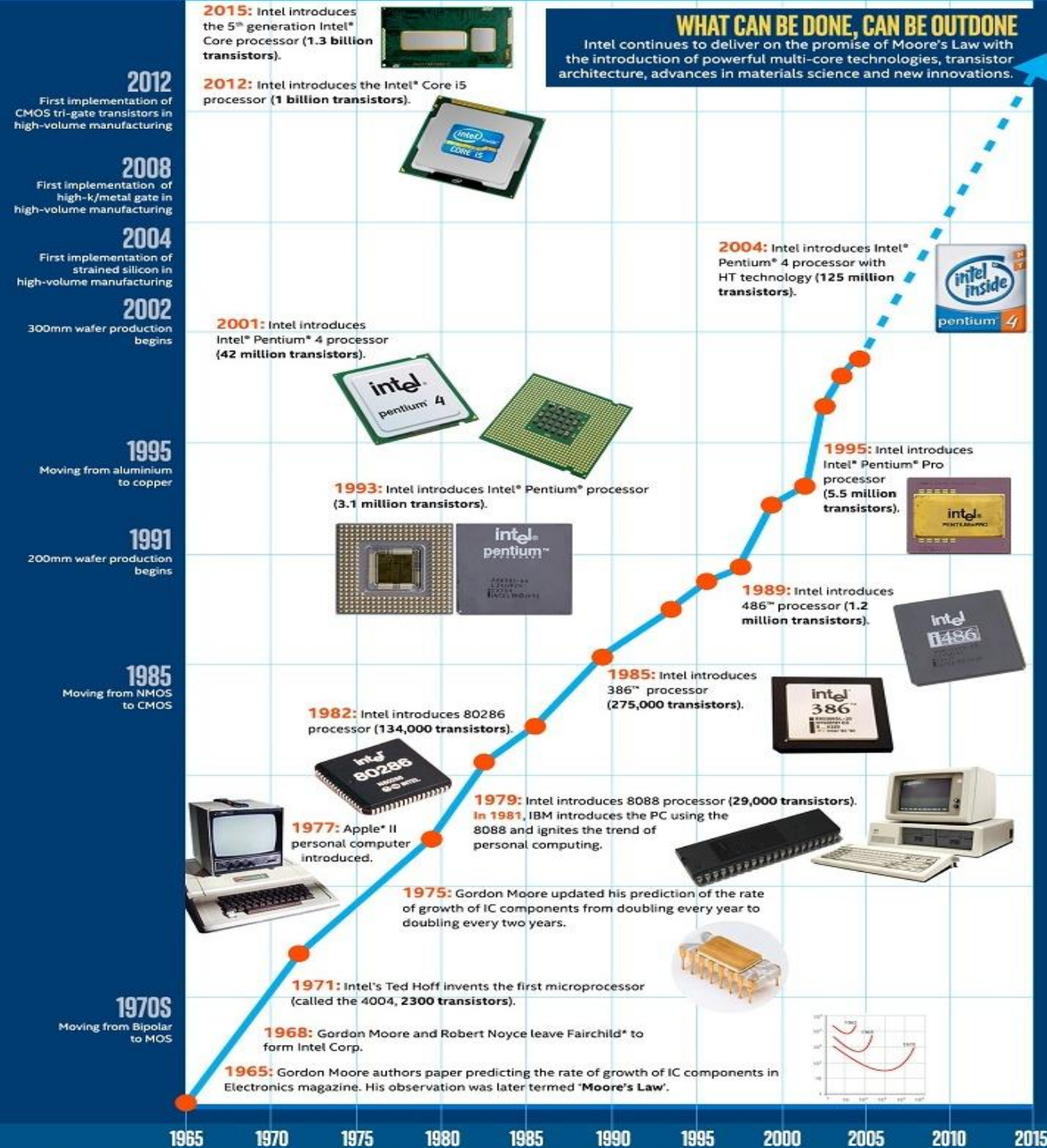
MOORE'S LAW TIMELINE

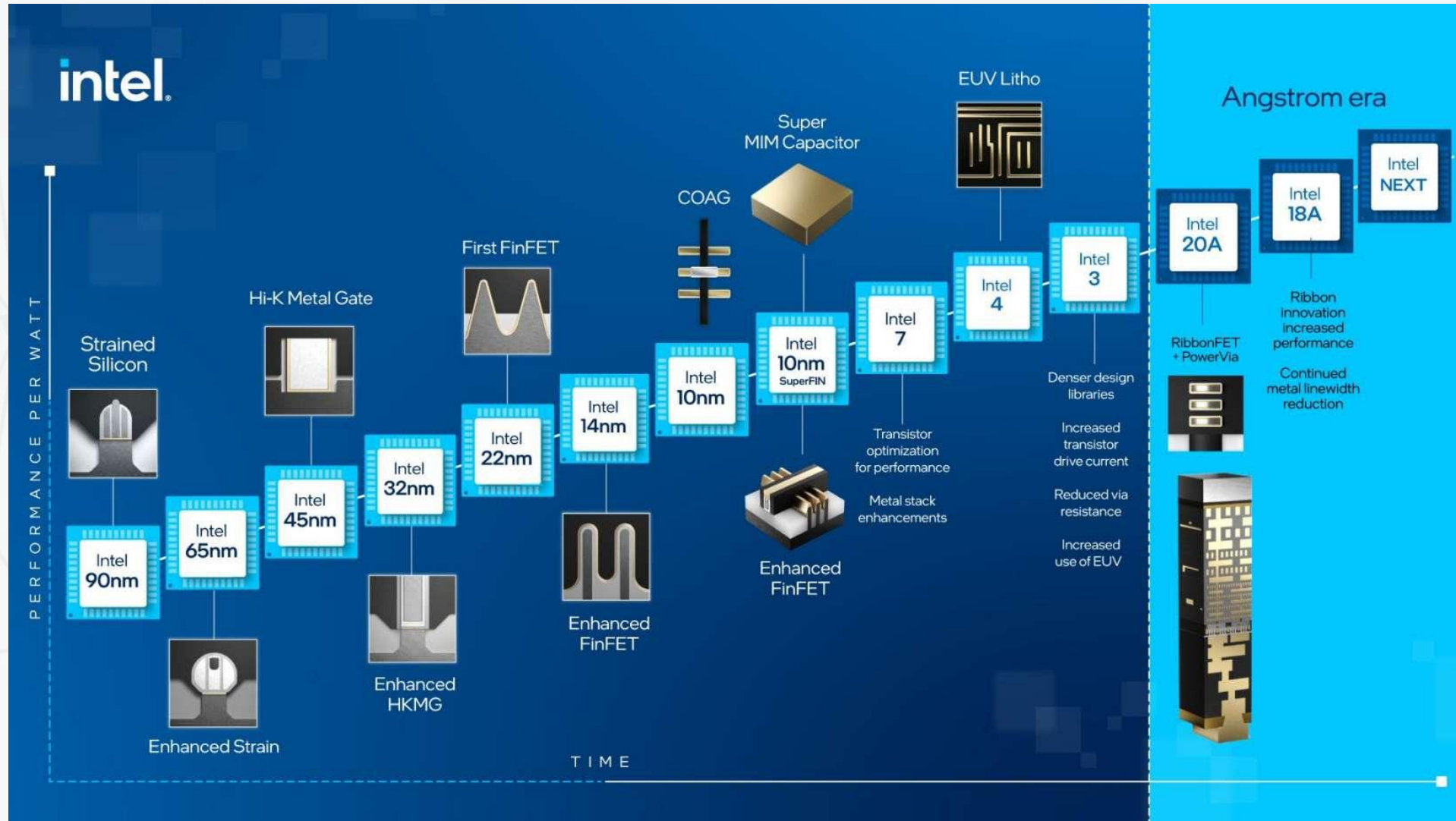
Moore's Law – the observation that computing dramatically decreases in cost at a regular pace – is short-hand for rapid technological change. Over the past 50 years, it has ushered in the dawn of the personalization of technology and enabled new experiences through the integration of technology into almost all aspects of our lives.





MOORE'S LAW
Moore's Law
at a regular p
50 years, it h
enabled new
all aspects o







REVOLUTION OF COMPUTER SYSTEMS [3]

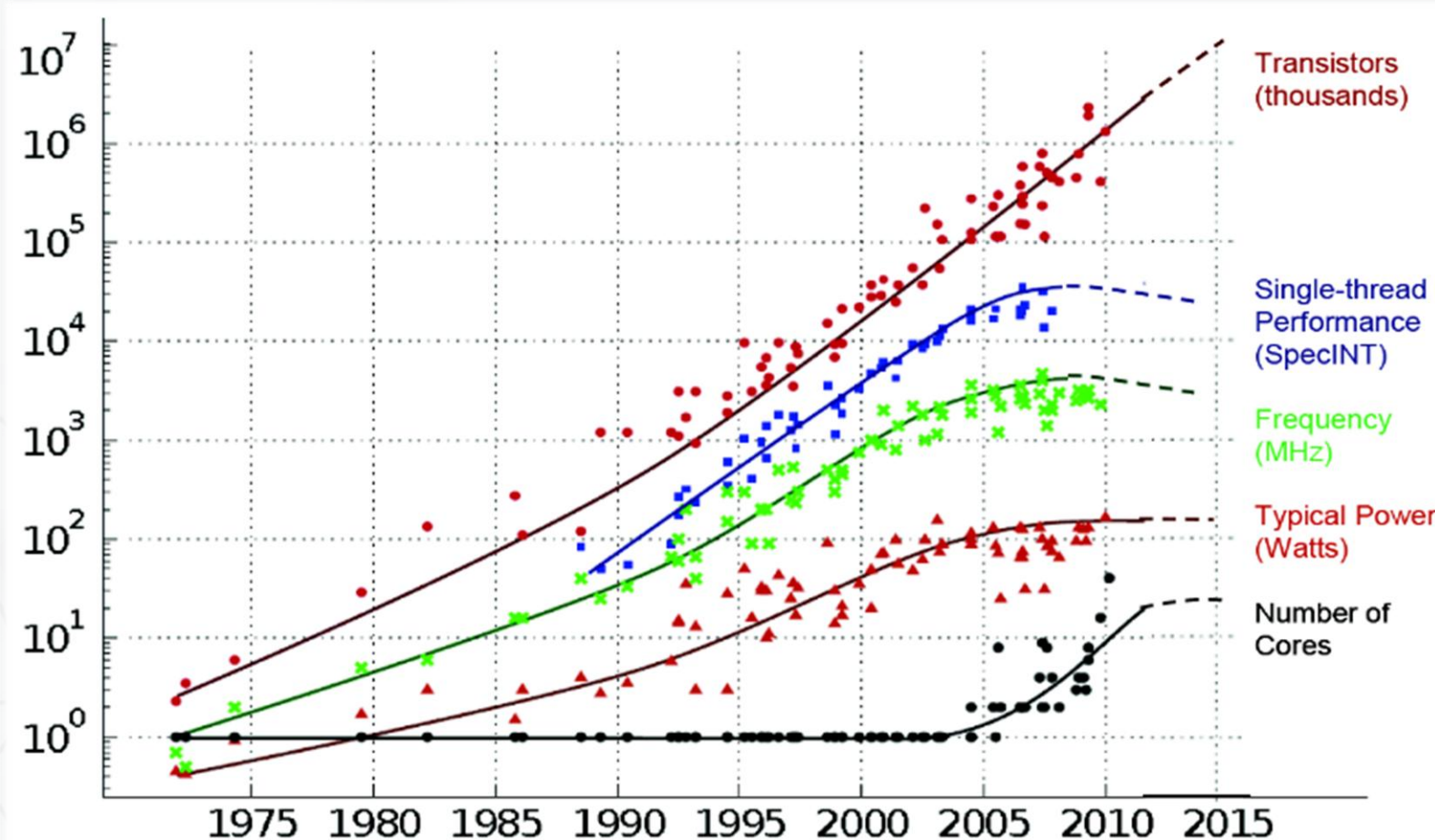
- 2nd development: high-speed computer networks
 - Ethernet was developed at Xerox PARC in 1973–1975 by Robert Metcalfe et al., commercially introduced in 1980, and standardized in 1983 as IEEE 802.3
 - Local-Area Networks (LANs) allow thousands of machines within a building to be connected in such a way that information can be transferred between machines at rates of billions of bits/second (bps)
 - Wide-Area Networks (WANs) allow millions of machines all over the earth to be connected at speed varying from tens of thousands to hundreds of millions bps



REVOLUTION OF COMPUTER SYSTEMS [4]

- The result of these technologies is that it is now not only *feasible*, but also *easy*, to put together computing systems composed of large numbers of computers, connected by a high-speed network
- In contrast to the previous centralized systems (or single-processor systems) consisting of a single computer, its peripherals, and perhaps some remote terminals, the current systems are usually networked computers or distributed systems (i.e., geographically dispersed)

THE FREE LUNCH IS OVER*



* Herb Sutter (2005); *The Free Lunch is Over: A Fundamental Turn toward Concurrency in Software*;
URL=<http://www.gotw.ca/publications/concurrency-ddj.htm>



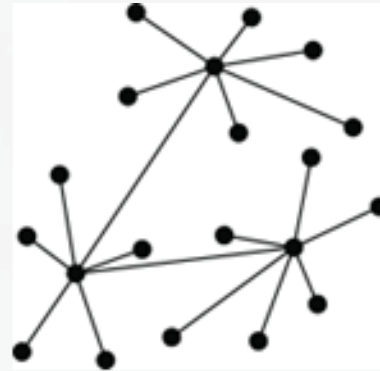
DISTRIBUTED SYSTEMS

DISTRIBUTED VS. DECENTRALIZED SYSTEMS *[1]*

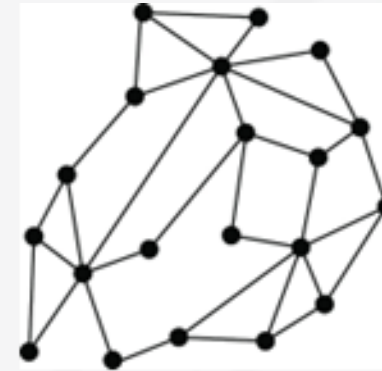
What many people state



Centralized



Decentralized



Distributed

- When does a decentralized system become distributed?
 - Adding 1 link between 2 nodes in a decentralized system?
 - Adding 2 links between 2 other nodes?
 - In general: adding $k > 0$ links ... ?



DISTRIBUTED VS. DECENTRALIZED SYSTEMS [2]

Alternative approach

- Two views on realizing a networked computer system
 - **Integrative view**: connecting existing networked computer systems into a larger system
 - **Expansive view**: an existing networked computer system is extended with additional computers
- Two definitions
 - A **decentralized system** is a networked computer system in which processes and resources are **necessarily** spread across multiple computers
 - A **distributed system** is a networked computer system in which processes and resources are **sufficiently** spread across multiple computers



DISTRIBUTED VS. DECENTRALIZED SYSTEMS [3]

- Decentralized systems are mainly related to the integrative view of networked computer systems; we want to connect systems, yet may be hindered by administrative boundaries
 - Many apps in the artificial-intelligence domain require massive amounts of data for building predictive models, but when data needs to stay within the perimeter of an organization, we need to bring the training to the data → **federated learning** (spreading processes and resources is dictated by administrative policies)
 - A **distributed ledger** (a.k.a., **blockchain**) is another example, in which processes and resources are *necessarily* spread across multiple computers due to lack of trust
 - Systems that are naturally geographically dispersed (systems in which an actual location, e.g., a power plant, a building, a specific natural environment, etc. needs to be monitored) → the necessity for spreading processes and resources comes from a spatial argument



DISTRIBUTED VS. DECENTRALIZED SYSTEMS [4]

- Distributed systems are mainly related to the expansive view of networked computer systems
 - **E-mail services** (e.g., Google Mail) require users to configure the incoming and outgoing servers (IMAP and SMTP), and with a massive number of users (e.g., close to 2B Google Mail users as of 2022), it is unlikely that only two servers can handle all their e-mails; behind the scenes, the entire Google Mail service has been implemented and spread across many computers, jointly forming a distributed system (that may expand or shrink as dictated by dependability requirements, in turn, dependent on the number of its users)
 - **Content Delivery Networks (CDNs for short)** copy and spread the content of an actual Website across various servers; when visiting a Website, the user is transparently redirected to a nearby server that holds all or part of the content of that Website (note: the content is not copied to all servers, yet only to where it makes sense, that is, *sufficiently*, and for the reasons of performance)



SOME COMMON MISCONCEPTIONS [1]

Centralized solutions do not scale

- Make distinction between **logically** and **physically** centralized
- The root of the Domain Name System:
 - logically centralized
 - physically (massively) distributed
 - decentralized across several organizations

Centralized solutions have a single point of failure

- Generally not true (e.g., the root of DNS)
- A single point of failure is often:
 - easier to manage
 - easier to make more robust



SOME COMMON MISCONCEPTIONS [2]

Important

- There are many, poorly founded, misconceptions regarding **scalability, fault tolerance, security**, etc.
- We need to develop skills by which distributed systems can be readily understood so as to judge such misconceptions



PERSPECTIVES ON DISTRIBUTED SYSTEMS

Distributed systems are complex and taking different perspectives:

- **Architecture**: common organizations
- **Process**: what kind of processes, and their relationships
- **Communication**: facilities for exchanging data
- **Coordination**: application-independent algorithms
- **Naming**: how do you identify resources?
- **Consistency and replication**: performance requires of data, which need to be the same
- **Fault tolerance**: keep running in the presence of partial failures
- **Security**: ensure authorized access to resources



DEFINITION OF A DISTRIBUTED SYSTEM [1]

- A collection of autonomous computing elements that appears to its users as a single coherent system
[van Steen and Tanenbaum]
 - Autonomous computing elements (i.e., nodes)
 - Users believe they are dealing with a single system
- What does it mean?
 - One way or the other the autonomous nodes need to collaborate; this is the heart of a distributed system
 - No assumptions are made concerning the type of nodes; they could range from high-performance mainframe computers to small nodes in sensor networks
 - No assumptions are made on the way that nodes are interconnected



DEFINITION OF A DISTRIBUTED SYSTEM [2]

- A system in which hardware and software components located at networked computers communicate and coordinate their actions only by passing messages [Coulouris et al.]
 - Concurrency of components:
Concurrent program execution is the norm, and resources are shared when necessary. The capacity of the system can be increased by adding more resources (e.g., computers) to the network.
 - No global clock:
Components coordinate their actions by exchanging messages, since there are limits to the accuracy with which the computers in a network can synchronize their clocks.



DEFINITION OF A DISTRIBUTED SYSTEM [3]

- A system in which ... [Coulouris et al.] (cont'd)
 - Independent failures of components:
It is the responsibility of system designers to plan for the consequences of possible failures. Distributed systems can fail in a few ways:
 - Faults in the network result in the isolation of the computers, but that does not mean that they stop running; in fact, the programs on them may not be able to detect whether the network has failed or has become unusually slow.
 - The failure of a computer or the unexpected termination of a program somewhere in the system (a crash) is sometimes not immediately made known to the other components with which it communicates. Each component of the system can fail independently, leaving the others still running.



EXAMPLES OF DISTRIBUTED SYSTEMS

- The World-Wide Web – information & resource sharing
- Web search (e.g., Google, Yahoo, Microsoft Bing)
- Massively multiplayer online games
- Financial trading
- Cluster computing (e.g., a network of workstations)
- Grid computing (e.g., collaboration of virtual organizations)
- Distributed manufacturing system (e.g., automated assembly line)
- Distributed information systems
- Sensor networks



MAIN CHARACTERISTICS OF DIST. SYSTEMS

- Differences between the various computers and the ways in which they communicate are mostly hidden from users
- Users and applications can interact with a distributed system in a consistent and uniform way, regardless of where and when interaction takes place
- Distributed systems should be relatively easy to expand or scale
- A distributed system will normally be continuously available, although perhaps some parts may be temporarily out of order



GOALS OF DISTRIBUTED SYSTEMS

- Supporting resource sharing
- Making distribution transparent
- Being open
- Being dependable
- Improving performance
- Being scalable



RESOURCE SHARING

- The main goal of a distributed system is to make it easy for the users (and applications) to access remote resources, and to share them in a controlled and efficient way
- Canonical examples
 - Cloud-based shared storage and files
 - Peer-to-peer assisted multimedia streaming
 - Shared mail services (think of outsourced mail systems)
 - Shared Web hosting (think of content distribution networks)
- Observation: *“The network is the computer”* (quote from John Gage, then at Sun Microsystems)
- Reasons for resource sharing:
 - Economic sense → sharing costly resources
 - Collaboration and exchanging information



TRANSPARENCY [1]

- What is transparency?

The phenomenon by which a distributed system attempts to hide the fact that its processes and resources are physically distributed across multiple computers, possibly separated by large distances

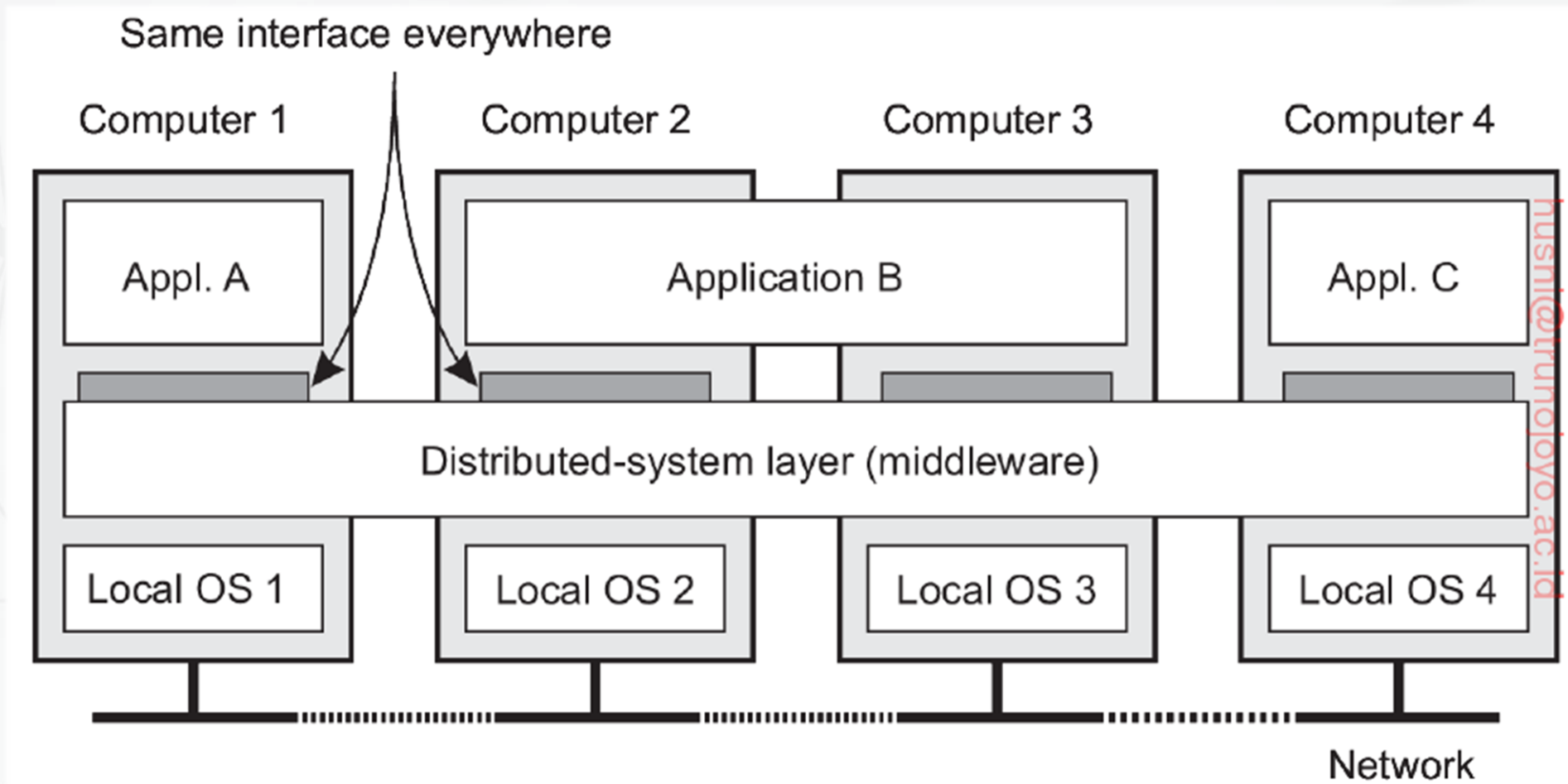
- Observation: Distribution transparency is handled through many different techniques in a layer between applications and operating systems → a **middleware layer**

- Some considerations:

- There is a trade-off between a high degree of transparency and the performance of a system, e.g., consistent vs. inconsistent replication
- Full distribution transparency is simply impossible; sometimes it may be much better to make distribution explicit so that the users and application developers understand the (sometimes unexpected) behavior and are better prepared to deal with it



MIDDLEWARE LAYER





TRANSPARENCY [2]

Different forms of transparency in a distributed system (note: an object can be a resource or a process)

Transparency	Description
Access	Hide difference in data representation & how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object



OPENNESS *[1]*

- An open distributed system is a system that offers components that can easily be used by, or integrated into other systems; an open distributed system itself will often consist of components that originate from elsewhere
- Be able to interact with services from other open systems, irrespective of the underlying environment:
 - Systems should conform to well-defined **interfaces**
 - Systems should easily **interoperate**
 - Systems should support **portability** of applications
 - Systems should be easily **extensible**



OPENNESS [2]

- Implementing openness: **policies**
 - What level of consistency do we require for client-cached data?
 - Which operations do we allow downloaded code to perform?
 - Which QoS requirements do we adjust in the face of varying bandwidth?
 - What level of secrecy do we require for communication?
- Implementing openness: **mechanisms**
 - Allow (dynamic) setting of caching policies
 - Support different levels of trust for mobile code
 - Provide adjustable QoS parameters per data stream
 - Offer different encryption algorithms



OPENNESS [3]

- Observation: The stricter the separation between policy and mechanism, the more we need to ensure proper mechanisms, potentially leading to many configuration parameters and complex management
- Finding a balance: Hard-coding policies often simplifies management and reduces complexity at the price of less flexibility → there is no obvious solution



DEPENDABILITY [1]

- To provide services to clients, the component may require the services from other components (i.e., a component may depend on some other component)
 - A component C depends on C^* if the correctness of C 's behavior depends on the correctness of C^* 's behavior (note: components are processes or channels)
- Requirements related to dependability

Requirement	Description
Availability	Readiness for usage
Reliability	Continuity of service delivery
Safety	Very low probability of catastrophes
Maintainability	How easy can a failed system be repaired



DEPENDABILITY [2]

- Reliability $R(t)$ of component C

Conditional probability that C has been functioning correctly during $[0, t)$ given C was functioning correctly at the time $T = 0$

- Traditional metrics

- Mean Time To Failure ($MTTF$): The average time until a component fails
- Mean Time To Repair ($MTTR$): The average time needed to repair a component
- Mean Time Between Failures ($MTBF$): Simply $MTTF + MTTR$



DEPENDABILITY [3]

- Failure vs. error vs. fault

Term	Description	Example
Failure	A component is not living up to its specifications	Crashed program
Error	Part of a component that can lead to a failure	Programming bug
Fault	Cause of an error	Sloppy programmer

- Handling fault

Term	Description	Example
Fault prevention	Prevent the occurrence of a fault	Don't hire sloppy programmers
Fault tolerance	Build a component and make it mask the occurrence of a fault	Build each component by two independent programmers
Fault removal	Reduce the presence, number, or seriousness of a fault	Get rid of sloppy programmers
Fault forecasting	Estimate current presence, future incidence, and consequences of faults	Estimate how a recruiter is doing when it comes to hiring sloppy programmers



DEPENDABILITY [4]

- Observation on security: A distributed system that is not secure, is not dependable
- What we need?
 - **Confidentiality**: info is disclosed only to authorized parties
 - **Integrity**: Ensure that alterations to assets of a system can be made only in an authorized way
- Authentication vs. authorization vs. trust
 - **Authentication**: verifying the correctness of a claimed id
 - **Authorization**: does an identified entity has proper access rights?
 - **Trust**: one entity can be assured that another will perform particular actions according to a specific expectation



PERFORMANCE

- Without gain on performance, why bother with distributed systems
- Performance loss due to communication delays
 - Fine-grain (tightly-coupled) parallelism → high degree of interaction
 - Coarse-grain (embarrassingly) parallelism → low degree of interaction
- Performance loss due to making the system fault tolerant, e.g., check-pointing, replication, migration



SCALABILITY [1]

- At least three components of scalability:
 - Size scalability → number of users or processes
 - Geographical scalability → max distance between nodes
 - Administrative scalability → number of administrative domains
- Observation: Most systems account only, to a certain extent, for size scalability (often a solution: multiple powerful servers operating independently in parallel)
- Today, the challenge still lies in geographical and administrative scalability



SCALABILITY [2]

- Root causes for size scalability problems with centralized solutions
 - The computational capacity, limited by the CPUs
 - The storage capacity, including the transfer rate between CPUs & disks
 - The network between the user and the centralized service
- Problems with geographical scalability
 - Cannot simply go from LAN to WAN: many distributed systems assume synchronous client-server interactions (latency may easily prohibit this scheme)
 - WAN links are often inherently unreliable: simply moving streaming video from LAN to WAN is bound to fail
 - Lack of multipoint communication, so that a simple search broadcast cannot be deployed (solution is to develop separate naming and directory services, which have their own scalability problems)



SCALABILITY [3]

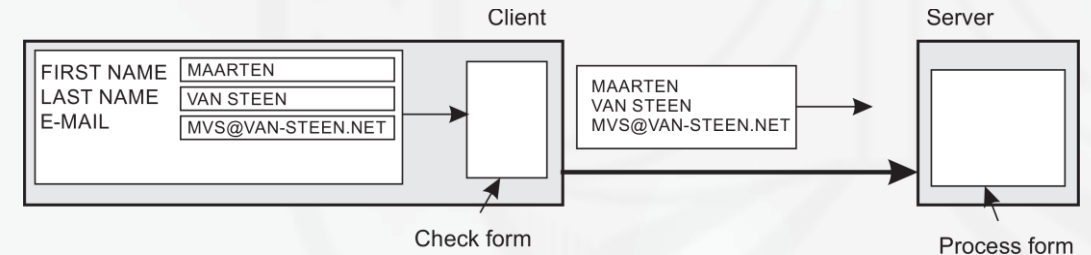
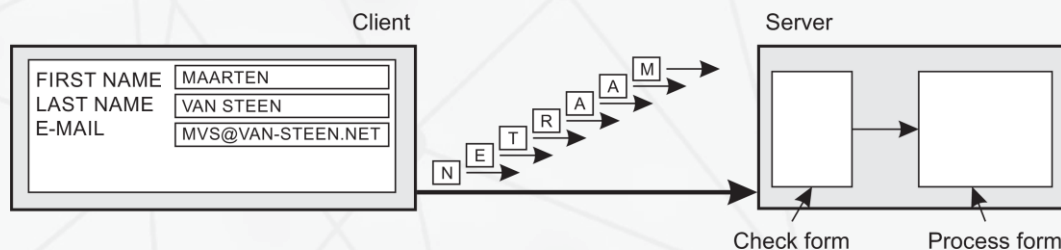
- Problems with administrative scalability
Conflicting policies concerning usage (and thus payment), management, and security
- Examples:
 - **Computational grids**: share expensive resources between different domains
 - **Shared equipment**: how to control, manage, and use a shared radio telescope constructed as large-scale shared sensor network?
- Exception: several peer-to-peer networks
 - File-sharing systems (based, e.g., on BitTorrent)
 - Peer-to-peer telephony (early versions of Skype)
 - Peer-assisted audio streaming (Spotify)



SCALABILITY [4]

Techniques for scaling

- Hide communication latencies
 - Make use of **asynchronous communication**
 - Have separate handler for incoming response
 - Problem: not every application fits this model
- Facilitate solution by moving computations to client





SCALABILITY [5]

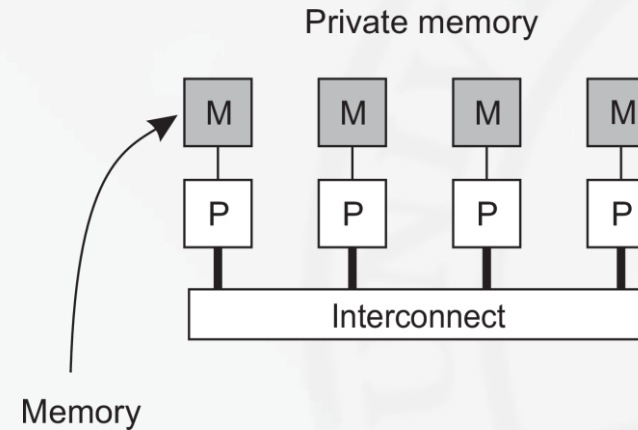
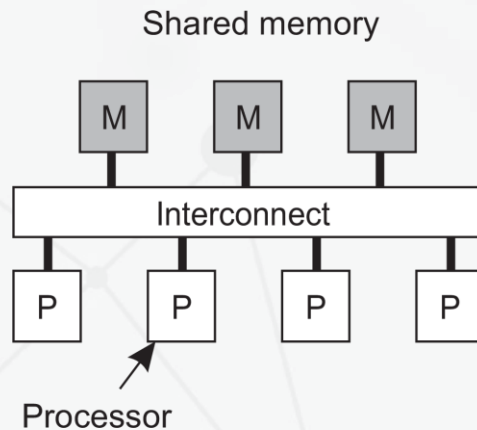
Techniques for scaling

- Partition data and computations across multiple machines
 - Move computations to clients (Java applets and scripts)
 - Decentralized naming services (DNS)
 - Decentralized information systems (WWW)
- Replication and caching: Make copies of data available at different machines
 - Replicated file servers and databases
 - Mirrored Websites
 - Web caches (in browsers and proxies)
 - File caching (at server and client)

CLASSIFICATION OF DISTRIBUTED SYSTEMS [1]

High-performance distributed computing

- Parallel computing
 - Multiprocessor vs. multicomputer architectures



- Multiprocessors are relatively easy to program in comparison to multicomputers, yet have problems when increasing the number of processors (or cores)
- Solution: try to implement a shared-memory model on top of a multicomputer



CLASSIFICATION OF DISTRIBUTED SYSTEMS [2]

High-performance distributed computing (cont'd)

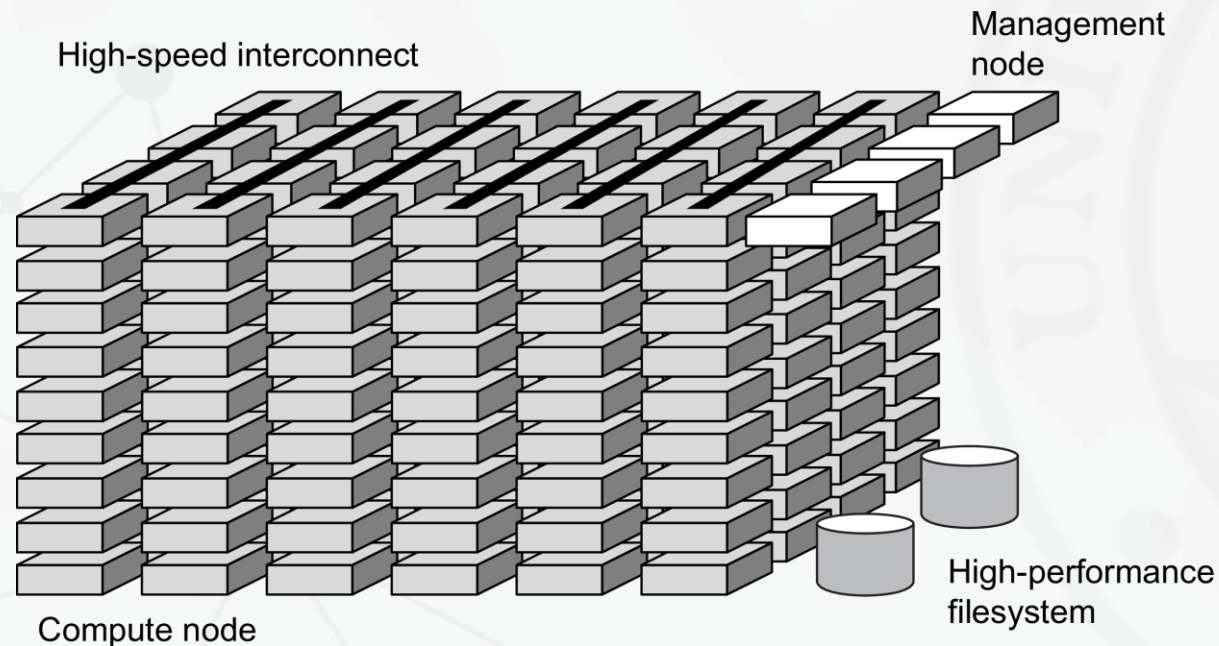
- Distributed shared-memory (DSM) systems
 - Virtual-memory techniques: map all main-memory pages (from different processors) into one single virtual address space
 - If a process at processor A addresses a page P located at processor B , the OS at A traps and fetches P from B , just as it would if P had been located on local disk
 - Problem: performance of distributed shared memory could never compete with that of multiprocessors and failed to meet the expectations of programmers (thus, it has been widely abandoned by now)



CLASSIFICATION OF DISTRIBUTED SYSTEMS [3]

High-performance distributed computing (cont'd)

- Cluster computing
 - Homogeneous: same OS, near-identical hardware
 - Single or tightly-coupled managing node(s)





CLASSIFICATION OF DISTRIBUTED SYSTEMS [4]

High-performance distributed computing (cont'd)

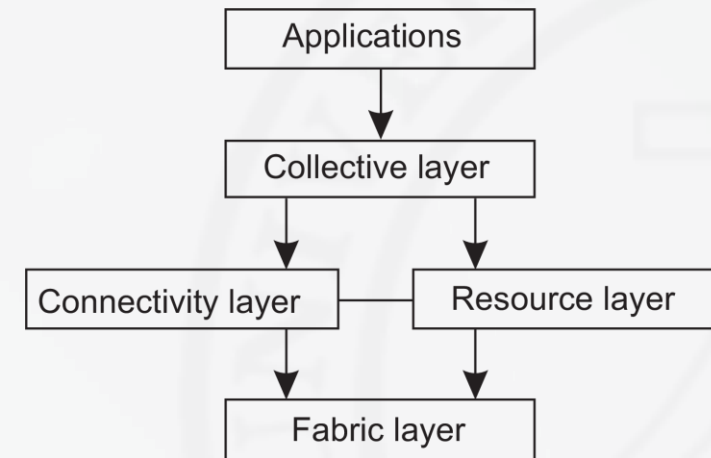
- Grid computing
 - The next step: plenty of nodes from everywhere
 - Heterogeneous
 - Dispersed across several organization
 - Can easily span a wide-area network
 - To allow for collaborations, grids generally use virtual organization; in essence, this is a grouping of users (or, their IDs) that allows for authorization on resource allocation
 - Architecture for grid computing
 - Fabric layer: provides interfaces to local resources (for querying state and capabilities, locking, etc.)
 - Connectivity layer: communication/transaction protocols, e.g., for moving data between resources (also various auth. protocols)



CLASSIFICATION OF DISTRIBUTED SYSTEMS [5]

High-performance distributed computing (cont'd)

- Grid computing (cont'd)
 - Architecture for grid computing (cont'd)
 - Resource layer: manages a single resource, such as creating processes or reading data
 - Collective layer: handles access to multiple resources (discovery, scheduling, replication)
 - Application layer: contains actual grid apps in a single organization





CLASSIFICATION OF DISTRIBUTED SYSTEMS [6]

Integrating applications

- Situation: organizations confronted with many networked applications, but achieving interoperability was painful
 - A networked application is one that runs on a server making its services available to remote clients
- Simple integration:
 - Clients combine requests for (different) applications
 - Send that off
 - Collect responses
 - Present a coherent result to the user
- Next step: allow direct application-to-application communication, leading to Enterprise Application Integration



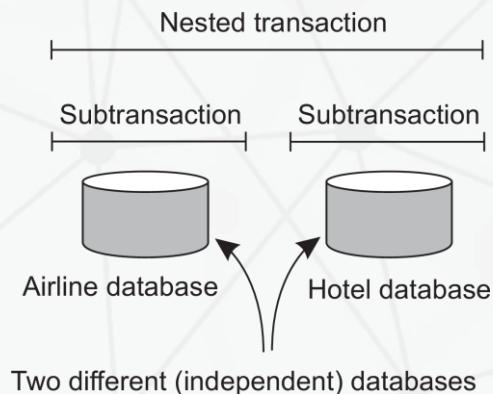
CLASSIFICATION OF DISTRIBUTED SYSTEMS [7]

Integrating applications (cont'd)

- Enterprise Application Integration

- Transaction

Primitive	Description
<i>BEGIN_TRANSACTION</i>	Mark the start of a transaction
<i>END_TRANSACTION</i>	Terminate the transaction and try to commit
<i>ABORT_TRANSACTION</i>	Kill the transaction and restore the old values
<i>READ</i>	Read data from a file, a table, or otherwise
<i>WRITE</i>	Write data to a file, a table, or otherwise



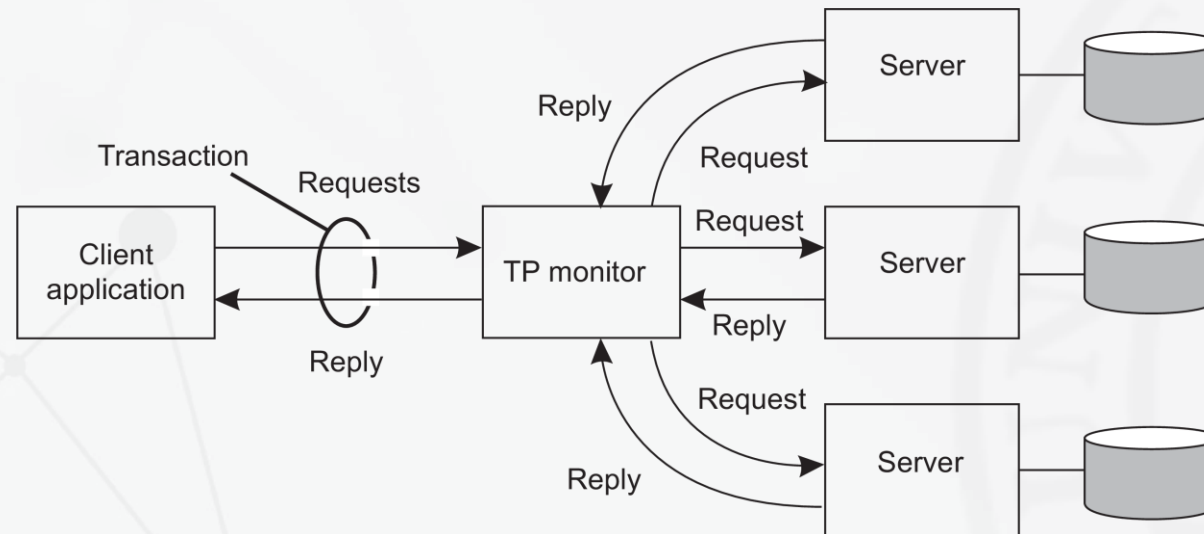
- Issue: all-or-nothing

- Atomic*: happens indivisibly (seemingly)
- Consistent*: does not violate system invariants
- Isolated*: not mutual interference
- Durable*: commit means changes are permanent

CLASSIFICATION OF DISTRIBUTED SYSTEMS [8]

Integrating applications (cont'd)

- Transaction Processing Monitor



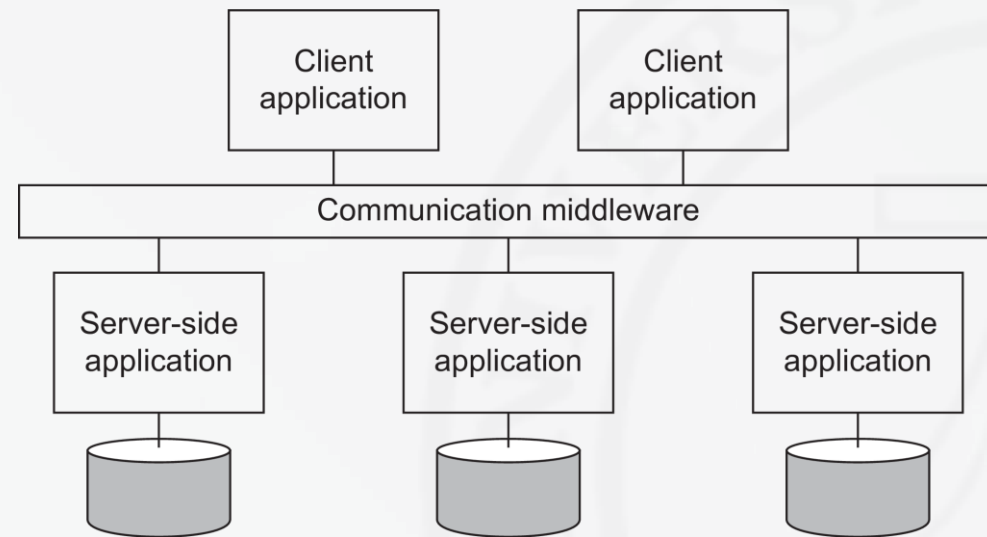
- Observation: often, the data involved in a transaction is distributed across several servers
- A TP monitor is responsible for coordinating the execution of a transaction



CLASSIFICATION OF DISTRIBUTED SYSTEMS [9]

Integrating applications (cont'd)

- Middleware and EAI
 - **Middleware** offers communication facilities for integration
 - **Remote Procedure Call (RPC)**: Requests are sent through local procedure call, packaged as message, processed, responded through message, and result returned as return from call
 - **Message-Oriented Middleware (MOM)**: Messages are sent to logical contact point (published) and forwarded to subscribed applications





CLASSIFICATION OF DISTRIBUTED SYSTEMS [10]

Integrating applications (cont'd)

How to integrate applications?

- **File transfer**: technically simple, but not flexible
 - Figure out the format and layout
 - Figure out file management
 - Update propagation (and consequently, update notifications)
- **Shared database**: much more flexible, but still requires common data schema, next to risk of bottleneck
- **Remote procedure call**: effective when execution of a series of actions is needed
- **Messaging**: RPCs require caller and callee to be up and running at the same time; messaging allow decoupling in time and space



CLASSIFICATION OF DISTRIBUTED SYSTEMS [11]

Distributed pervasive systems

- Observation: emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system, characterized by the fact that the system naturally blends into the user's environment
- Three (overlapping) subtypes
 - **Ubiquitous computing systems**: pervasive and continuously present, i.e., there is a continuous interaction between system and user
 - **Mobile computing systems**: pervasive, but emphasis is on the fact that devices are inherently mobile
 - **Sensor (and actuator) networks**: pervasive, with emphasis on the actual (collaborative) sensing and actuation of the environment



CLASSIFICATION OF DISTRIBUTED SYSTEMS [12]

Distributed pervasive systems (cont'd)

- Ubiquitous systems

Core elements:

- (**distribution**) Devices are networked, distributed, and accessible transparently
- (**interaction**) Interaction between users and devices is highly unobtrusive
- (**context awareness**) The system is aware of a user's context to optimize interaction
- (**autonomy**) Devices operate autonomously without human intervention, and are thus highly self-managed
- (**intelligence**) The system as a whole can handle a wide range of dynamic actions and interactions



CLASSIFICATION OF DISTRIBUTED SYSTEMS [13]

Distributed pervasive systems (cont'd)

- Mobile computing

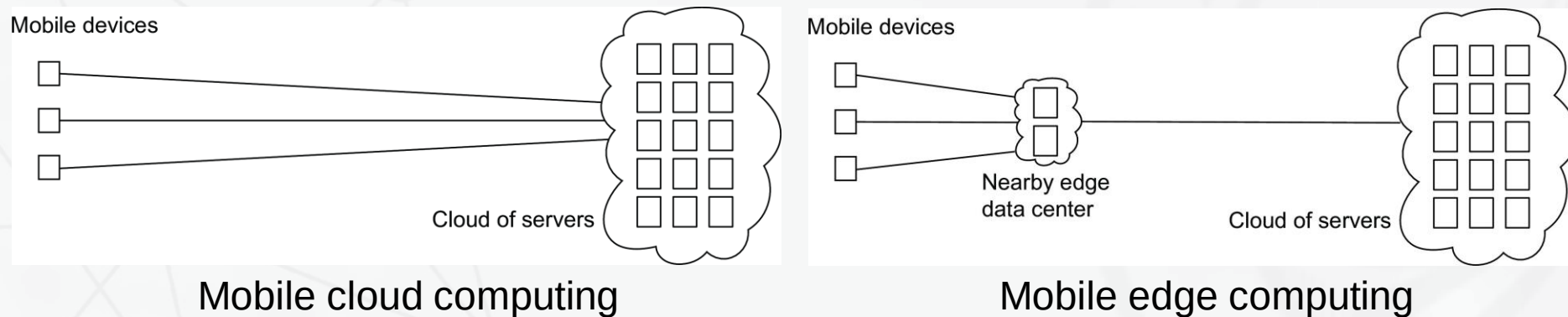
Distinctive features:

- A myriad of different mobile devices (smartphones, tablets, GPS devices, remote controls, active badges)
- Mobile implies that a device's location is expected to change over time → change of local services, reachability, etc. (keyword: discovery)
- Maintaining stable communication can introduce serious problems
- For a long time, research has focused on directly sharing resources between mobile devices; it never became popular and is by now considered to be a fruitless path for research

CLASSIFICATION OF DISTRIBUTED SYSTEMS [14]

Distributed pervasive systems (cont'd)

- Mobile computing (cont'd)
 - Bottomline: mobile devices set up connections to stationary servers, essentially bringing mobile computing in the position of clients of cloud-based services



CLASSIFICATION OF DISTRIBUTED SYSTEMS [15]

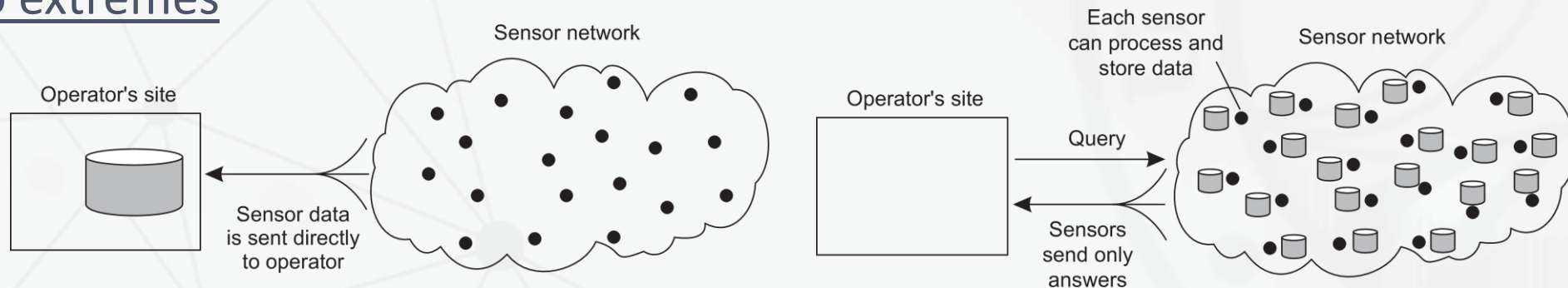
Distributed pervasive systems (cont'd)

- Sensor networks

The nodes to which sensors are attached are:

- Many (10s-1000s)
- Simple (small memory/compute/communication capacity)
- Often battery-powered (or even battery-less)

Two extremes

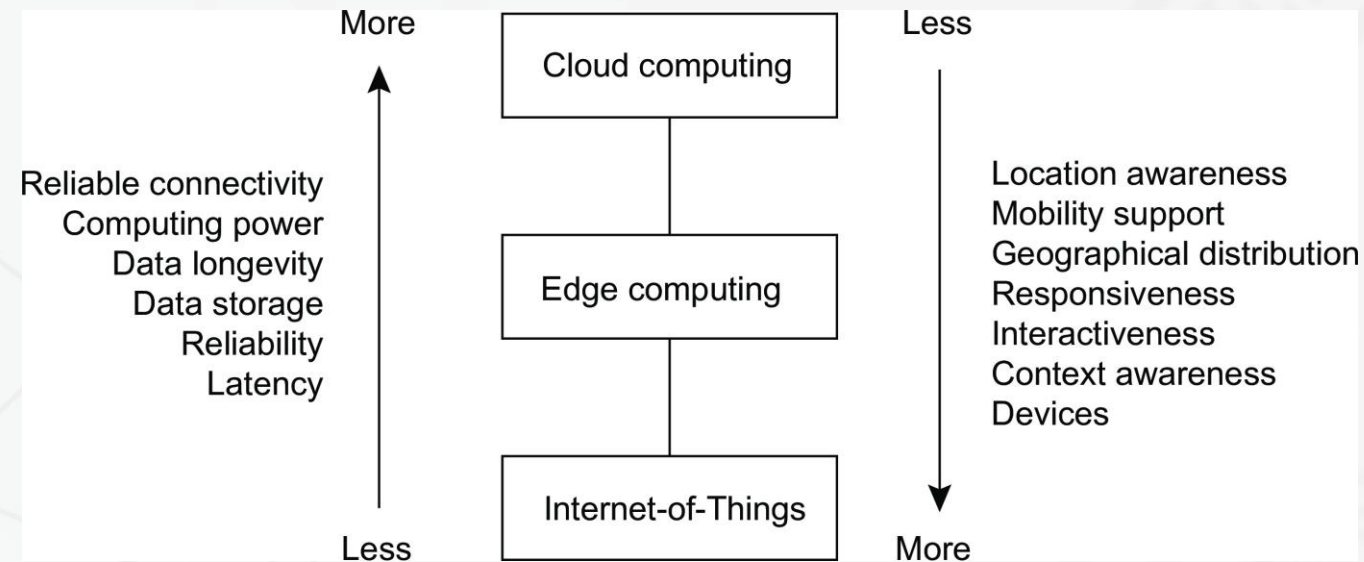




CLASSIFICATION OF DISTRIBUTED SYSTEMS [16]

Distributed pervasive systems (cont'd)

- Cloud-edge continuum





PITFALLS

- Distributed systems differ from traditional software because components are dispersed across a network
- False assumptions that everyone makes when developing a distributed application for the first time:
 - The network is reliable
 - The network is secure
 - The network is homogeneous
 - The topology does not change
 - Latency is zero
 - Bandwidth is infinite
 - Transport cost is zero
 - There is one administrator
- Unique properties of distributed systems: reliability, security, heterogeneity, network topology, latency & bandwidth, transport costs, and administrative domains

World's Fastest Computer (by June 2026)

Source: <https://top500.org>



- Processors : LX2 304C 1.55GHz (based on ARMv9)
- Accelerators : --
- No. of nodes : 22,680 (@ 2 LX2 CPU) → ±90 compute cabinets
- Total cores : 13,789,440
- Memory : 11,340 TB DDR5 RAM (+ 1,417.5 TB High-Bandwidth Memory)
- Interconnect : LingQi (1.6 Tb/s per node)
- Linpack performance : 2,198.40 PFlop/s (peak = 2,735.82 PFlop/s)
- Power : 42,220.00 kW
- OS : Kylin OS
- Parallel library : OpenMPI with customized LingQi interconnect

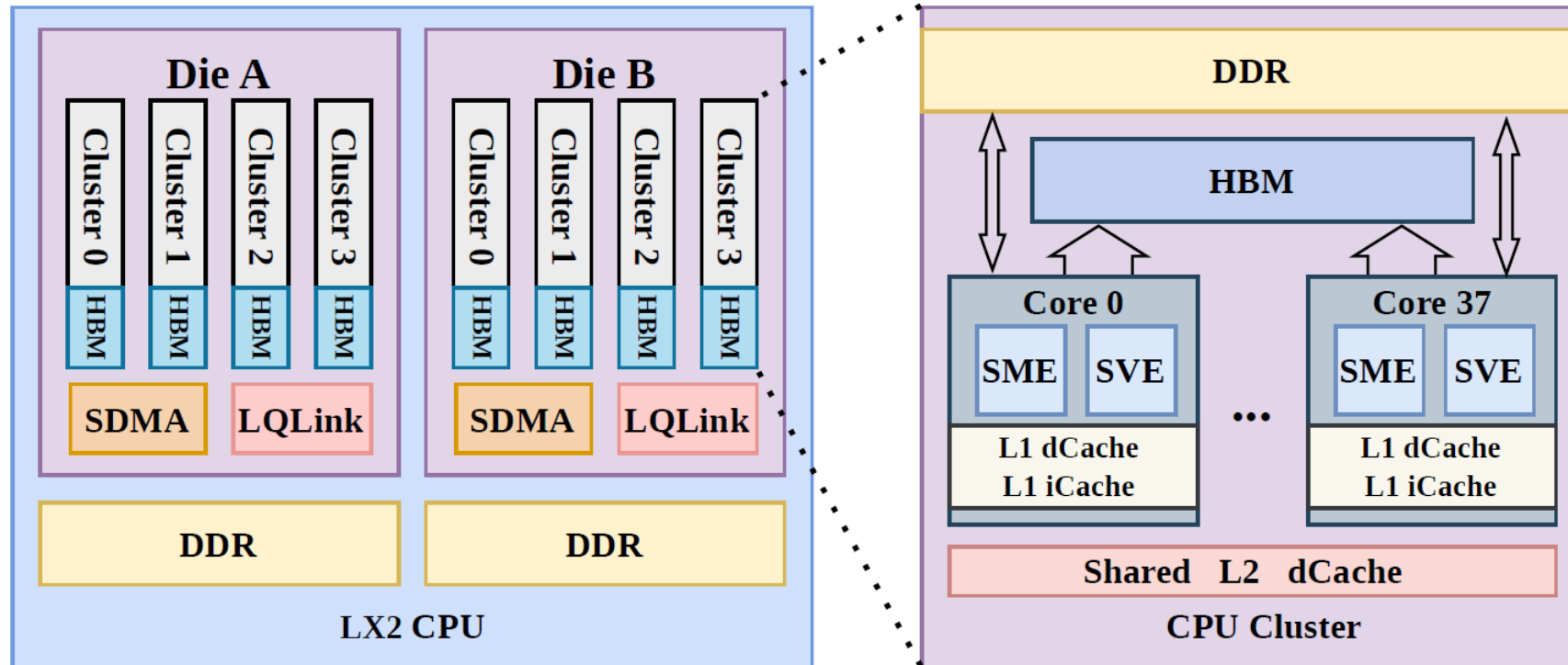
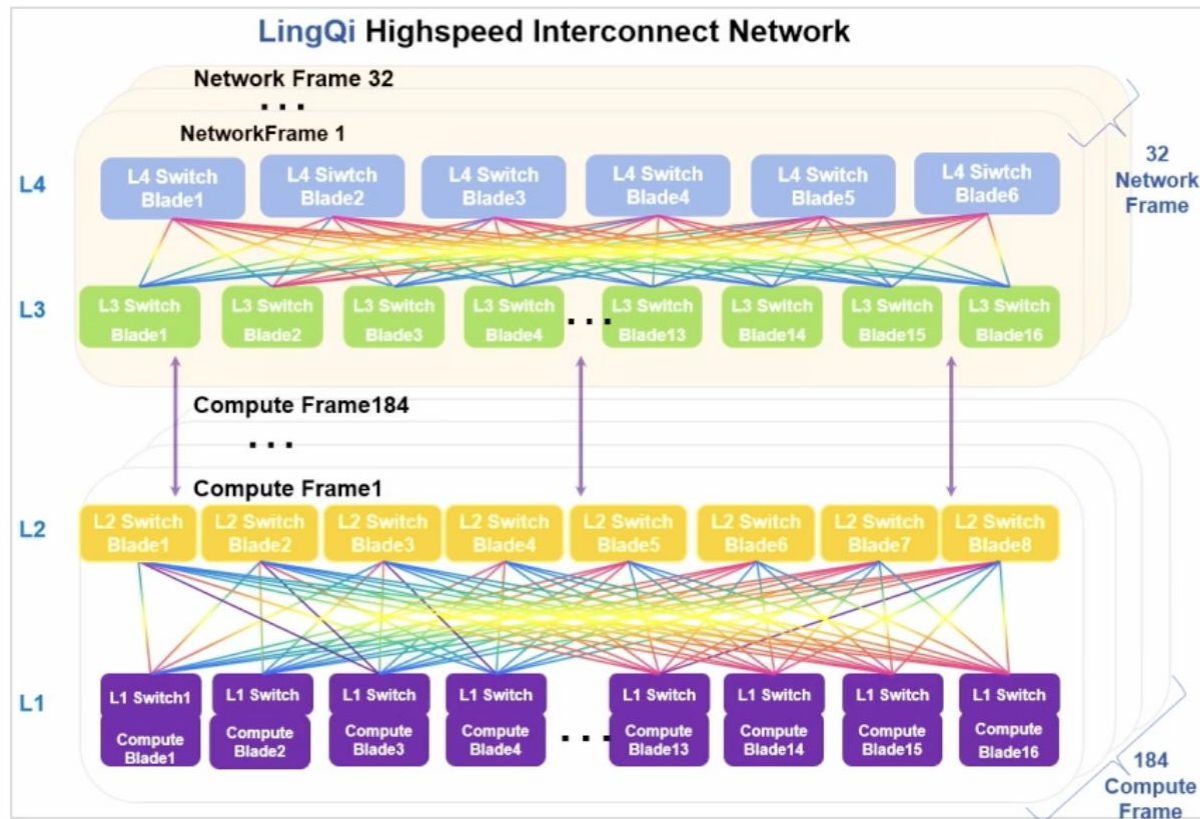


Figure 1: Architectural overview of the LX2 processor.

Interconnect Network



Architecture Capabilities

● High Performance & Scalability

- 4-layer fat-tree topology
- Large-scale network: 100K nodes, 2M ports
- Bisection bandwidth: ≥ 3.5 Pbps
- Minimum latency: $1.07 \mu s$

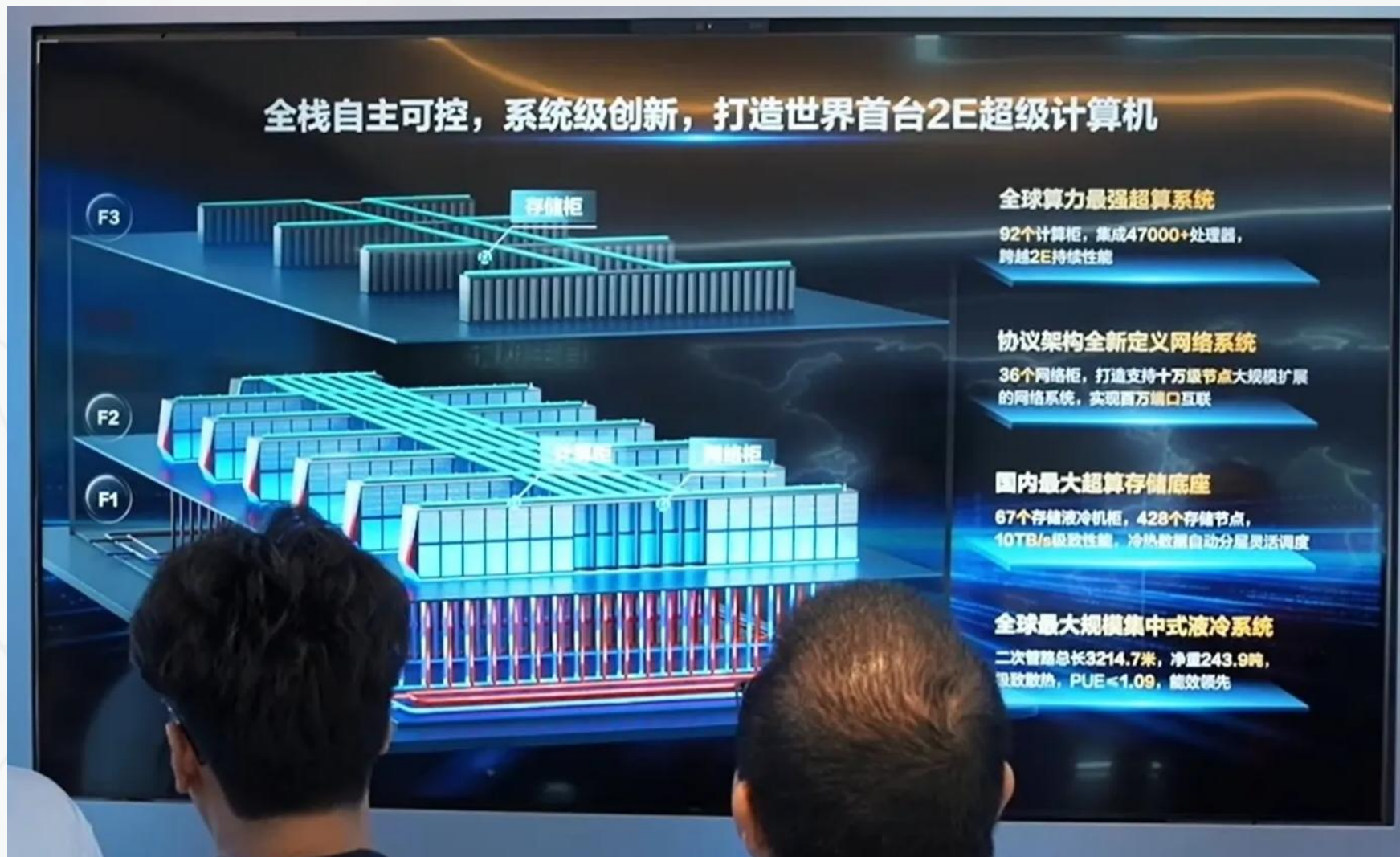


● High Reliability

- Dual-plane networking with multi-rail communication
- Credit-based flow control for lossless communication
- Four-layer network with only one optical layer
- Link-, chip-, and cabinet-level redundancy

● High Maintainability

- Hardware-supported telemetry with second-level data collection and proactive push



TOP500.ORG (JUNE 2026) *[1]*



Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	LineShine - LingKun, LX2 304C 1.55GHz, LingQi, Kylin OS, Shenzhen Cloud Computing Center Co., Ltd. National Supercomputing Centre in Shenzhen (NSCS) China	13,789,440	2,198.40	2,735.82	42,220
2	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
3	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
4	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
5	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, Bull EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794



TOP500.ORG (JUNE 2026) [2]

- **LineShine** (NSCS, China) is the TOP500 No. 1 supercomputer, achieving **2.198 Eflop/s** on HPL with 13,789,440 cores. Built on the LingKun platform with LX2 304C processors and LingQi interconnect, it also leads HPCG with 22.00 Petaflop/s.
- **El Capitan** (LLNL, USA) is the TOP500 No. 2 supercomputer, reaching **1.809 Eflop/s** on HPL with 11,340,000 cores. Powered by AMD 4th Gen EPYC CPUs and MI300A accelerators, it achieves 60.94 Gigaflops/watt and ranks No. 2 on HPCG.
- **Frontier** (ORNL, USA) is the TOP500 No. 3 supercomputer, achieving **1.353 Eflop/s** using 9,066,176 cores. Based on the HPE Cray EX architecture, it combines 3rd Gen AMD EPYC CPUs, AMD Instinct MI250X accelerators, and Slingshot-11 networking.
- The entry level to the list moved up to the **2.66 Pflop/s** mark on the Linpack benchmark. The last system on the newest list was listed at position **476** in the previous TOP500.
- Total combined performance of all 500 exceeded the Exaflop barrier with now **18.73 Eflop/s** up from 14.99 Eflop/s 6 months ago.
- The average concurrency level in the TOP500 is **305,404 cores per system** up from 270,522 six months ago.



TOP500.ORG (JUNE 2026) [3]

Country	Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
United States	161	32.2	7,031,439	11,225,198	56,392,340
Japan	44	8.8	1,517,868	1,952,195	13,775,616
Germany	41	8.2	1,402,709	1,828,639	8,921,180
China	31	6.2	2,377,226	3,027,843	26,606,264
France	21	4.2	317,095	491,265	4,406,752
South Korea	19	3.8	605,504	764,456	3,838,364
Italy	18	3.6	1,450,041	1,996,101	10,882,304
Canada	17	3.4	153,010	233,022	1,869,408
United Kingdom	11	2.2	447,660	614,634	2,862,120
Taiwan	11	2.2	213,778	305,704	1,089,664



TOP500.ORG (JUNE 2026) [4]

Manufacturer	Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
Lenovo	130	26	880,720	1,445,888	14,190,972
HPE	124	24.8	7,995,766	11,430,531	55,604,996
Bull	58	11.6	1,997,709	2,666,024	16,280,280
DELL	49	9.8	883,448	1,480,312	6,657,632
Nvidia	36	7.2	984,188	1,377,173	4,634,880
NEC	15	3	113,209	147,073	935,288
Fujitsu	13	2.6	664,644	831,679	9,789,432
Microsoft Azure	8	1.6	757,160	1,127,999	3,540,480
MEGWARE	8	1.6	82,172	127,643	564,464
Supermicro	8	1.6	213,229	308,297	1,256,280



TOP500.ORG (JUNE 2026) *[5]*

Interconnect	Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
Infiniband	292	58.4	7,097,007	10,079,678	45,721,524
Gigabit Ethernet	165	33	8,538,848	13,113,105	68,424,116
Omnipath	26	5.2	155,895	214,703	3,160,252
Proprietary Network	7	1.4	524,222	633,235	8,994,816
Custom Interconnect	6	1.2	2,335,610	2,927,960	26,075,704
Ethernet	4	0.8	81,161	106,539	325,632



TOP500.ORG (JUNE 2026) [6]

Processor	Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
Intel Sapphire Rapids	84	16.8	3,678,943	5,945,363	21,735,160
Intel Cascade lake	69	13.8	328,855	588,737	7,312,120
AMD Zen-3 (Milan)	66	13.2	2,796,119	4,037,140	25,192,976
AMD Zen-2 (Rome)	54	10.8	650,893	912,012	10,411,376
AMD Zen-4 (Genoa)	51	10.2	3,359,676	5,151,322	21,886,072
Intel Emerald Rapids	34	6.8	1,199,087	1,775,460	5,359,080
Intel Skylake	33	6.6	159,158	248,548	3,052,244
Intel Ice Lake	31	6.2	412,413	593,300	4,864,668
NVIDIA Grace CPU	25	5	2,507,534	3,268,167	12,250,896
AMD Zen-5 (Turin)	20	4	493,687	629,077	3,571,176



TOP500.ORG (JUNE 2026) [7]

Accelerator/ Co-processor	Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
NVIDIA H200 SXM5 141 GB	26	5.2	1,056,997	1,632,010	3,754,040
NVIDIA H100 SXM5 80GB	26	5.2	874,372	1,318,428	3,305,456
NVIDIA A100	23	4.6	309,891	470,789	3,190,960
NVIDIA GH200 Superchip	21	4.2	2,449,899	3,203,773	11,925,312
NVIDIA H100	17	3.4	939,587	1,428,714	3,215,984
NVIDIA A100 SXM4 40 GB	14	2.8	175,658	244,860	1,785,304
Nvidia H100 SXM5 94Gb	14	2.8	219,048	345,378	894,400
NVIDIA Tesla V100	13	2.6	96,292	143,823	1,747,944
AMD Instinct MI250X	13	2.6	2,396,008	3,452,065	16,293,760
AMD Instinct MI300A	13	2.6	2,804,049	4,265,429	17,122,392